สรุปและวิเคราะห์ผล SSIS ETL Benchmark

Created	@March 4, 2026 5:45 AM
Tags	

จากการทดสอบและจับเวลาได้ไฟล์ที่สามารถสรุปเป็นตารางได้ดังนี้:

dataset	extract_sec	transform_sec	load_sec
10k	0.163	2.484	0.013
100k	0.707	43.221	0.077
1m	9.991	449.141	0.029

เมื่อนำข้อมูลดังกล่าวมาวิเคราะห์และสร้างภาพด้วย MATLAB เพื่อให้เห็นภาพได้ชัดเจน ได้นำเสนอกราฟ 3 รูปแบบ ได้แก่ Grouped Bar Chart, Stacked Bar Chart และ Line Chart ตามลำดับ ดังแสดงในรูปที่ 1–3

1. การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล (Grouped Bar Chart)

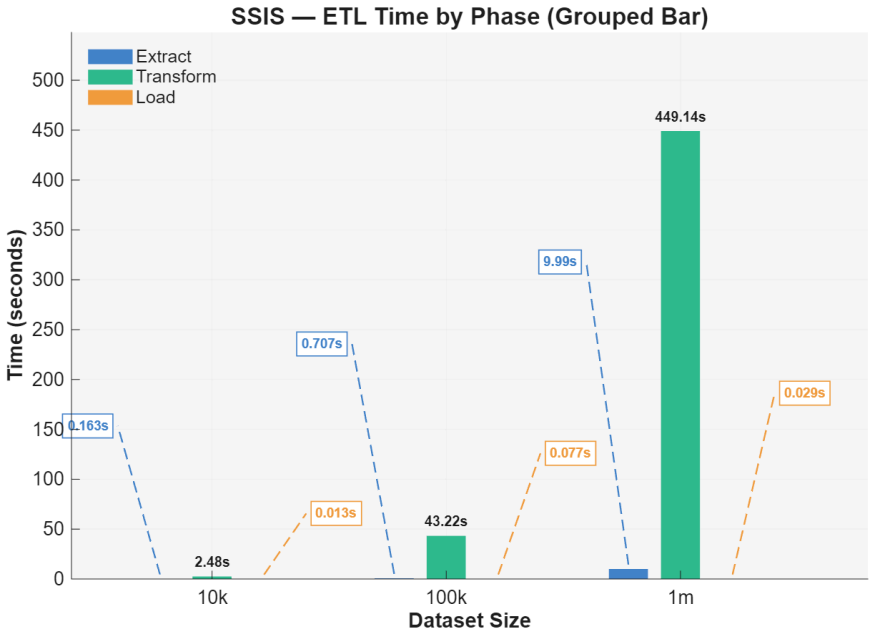

Fig 1 : การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล

จากภาพ Fig 1 : การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล เป็นภาพที่แสดงการเปรียบเทียบเวลาแต่ละ Phase (Extract / Transform / Load) แยกตาม Dataset Size จากกราฟพบว่า Transform Phase ใช้เวลามากที่สุดในทุก Dataset Size อย่างเห็นได้ชัด โดย Extract และ Load แทบมองไม่เห็นในสเกลเดียวกับ Transform เมื่อข้อมูลขนาดใหญ่ขึ้น สังเกตได้ว่า Transform ที่ 1m rows ใช้เวลา 449.14 วินาที ในขณะที่ Extract และ Load รวมกันใช้เพียง 10.02 วินาทีเท่านั้น จากการวิเคราะห์ใน Transform Phase สาเหตุมาจากการใช้ Script Component เป็นการรันแบบ Synchronous (ต้องรอคำสั่งก่อนหน้าทำงานเสร็จก่อนถึงจะรันคำสั่งต่อไปได้) ในเอกสารของ Microsoft ระบุว่า `ProcessInputRow()` ถูกเรียกหนึ่งครั้งต่อหนึ่ง row **[1]** ทำให้การ Hash จำนวน 4 columns สำหรับ 1m rows คือการคำนวณ hash รวม **4,000,000 ครั้ง** ด้วยสาเหตุใน Transform Phase จึงใช้เวลานานกว่า Extract และ Load ดังนั้น

---

2. สัดส่วนของแต่ละ Phase ต่อเวลารวม (Stacked Bar Chart)

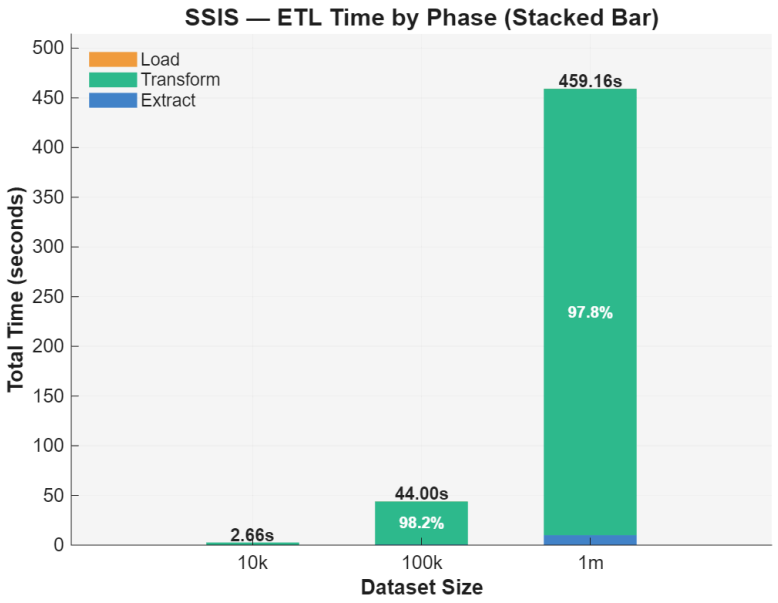

Fig 2 : สัดส่วนของแต่ละ Phase ต่อเวลารวม

จากภาพ Fig 2 สะท้อนให้เห็นว่า Transform Phase เป็น Bottleneck(คอขวด หรือ ข้อจำกัด) แม้ใน Dataset Size ที่ 10k จะไม่แสดงใน bar เพราะ bar เล็กเกินไปแต่เราสามารถคำนวณได้โดยใช้

วิธีคิด:

$$\text{Phase\%} = (\text{Phase_sec} \div \text{Total_sec}) \times 100$$

Total_sec = Extract_sec + Transform_sec + Load_sec

ทำให้เมื่อคำนวณสำหรับ Transform Phase สามารถคำนวณได้ดังนี้

```
10k rows
Total = 0.163 + 2.484 + 0.013 = 2.660s
Extract%   = (0.163 ÷ 2.660) × 100 = 6.13%
Transform% = (2.484 ÷ 2.660) × 100 = 93.38%
Load%      = (0.013 ÷ 2.660) × 100 = 0.49%

100k
Total = 0.707 + 43.220 + 0.077 = 44.004s
Extract%   = (0.707  ÷ 44.004) × 100 = 1.61%
Transform% = (43.220 ÷ 44.004) × 100 = 98.22%
Load%      = (0.077  ÷ 44.004) × 100 = 0.18%

1m rows
Total = 9.991 + 449.141 + 0.029 = 459.161s
Extract%   = (9.991   ÷ 459.161) × 100 = 2.18%
Transform% = (449.141 ÷ 459.161) × 100 = 97.82%
Load%      = (0.029   ÷ 459.161) × 100 = 0.01%
```

จากการคำนวณเห็นได้ว่า 97.8% ของเวลาทั้งหมดหายไปกับ Transform Phase เพียง Phase เดียว โดยที่ Dataset Size ที่ 10k ใช้เวลา Transform ไปถึง **93%** ของเวลา , 100k ใช้เวลา Transform ไปถึง **98.2%** ของเวลา และที่ 1m ใช้เวลา Transform ไปถึง **97.8%** ของเวลา จึงสรุปได้ว่า Transform Phase เป็น **ขั้นตอนที่ใช้เวลามากที่สุดและส่งผลโดยตรงต่อเวลารวมทั้งหมด**

---

3. อัตราการขยายตัวของแต่ละ Phase (Scalability)

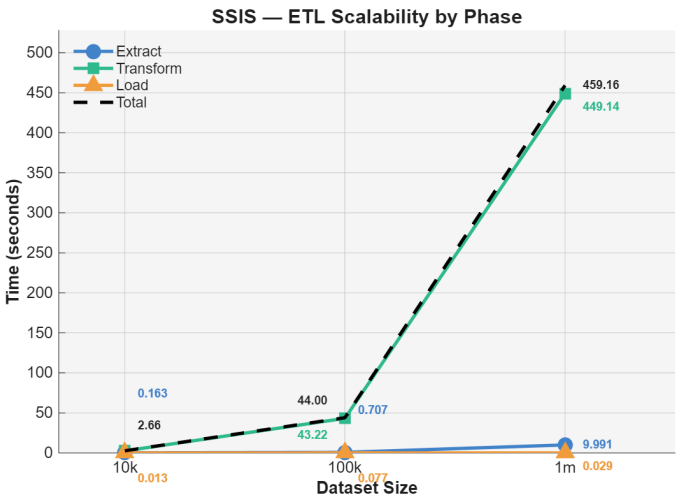

Fig 3 : อัตราการขยายตัวของแต่ละ Phase (Scalability)

จากรูปที่ 3 คำนวณ Scaling Ratio (อัตราส่วนการปรับขนาด) ด้วยสูตร

Scaling Ratio = เวลา Batch ถัดไป ÷ เวลา Batch ปัจจุบัน

ได้ผลดังนี้

```
Extract Scaling:
10k → 100k: 0.707 / 0.163 = 4.34x  (ข้อมูลเพิ่ม 10x หรือ 10 เท่า)
100k → 1m:  9.991 / 0.707 = 14.13x (ข้อมูลเพิ่ม 10x)

Transform Scaling:
10k → 100k: 43.220 / 2.484  = 17.40x (ข้อมูลเพิ่ม 10x)
100k → 1m:  449.141 / 43.220 = 10.39x (ข้อมูลเพิ่ม 10x)
10k → 1m:   449.141 / 2.484  = 180.81x (ข้อมูลเพิ่ม 100x 100 เท่า เวลาเพิ่มถึง 180.81 เท่า)

Load Scaling:
10k → 100k: 0.077 / 0.013 = 5.92x (ข้อมูลเพิ่ม 10x)
100k → 1m:  0.029 / 0.077 = 0.38x (เร็วขึ้น → Buffer/cache effect)
10k → 1m:   0.029 / 0.013 = 2.23x

Total Scaling:
```

```
10k → 100k: 44.004 / 2.660  = 16.54x
100k → 1m:  459.161 / 44.004 = 10.44x
```

จากตัวเลขข้างต้น สามารถอธิบายแต่ละขั้นตอนได้ดังนี้

ขั้นตอนดึงข้อมูล (Extract) ช่วง 10k→100k ใช้เวลาเพิ่มขึ้น 4.34 เท่า ช่วง 100k→1m เวลาเพิ่มขึ้นถึง 14.13 เท่า ซึ่ง Microsoft ระบุว่า disk เป็นการดำเนินการที่ใช้ทรัพยากรมากและใช้เวลานานกว่าการอ่านจาก Memory **[2]** ดังนั้นเมื่อข้อมูลมีขนาดใหญ่ขึ้นมาก การอ่านข้อมูลจาก Disk จึงส่งผลให้เวลา Extract เพิ่มขึ้นอย่างชัดเจน

ขั้นตอนแปลงข้อมูล (Transform) ช่วง 10k→100k เวลาเพิ่มขึ้นถึง 17.40 เท่า เนื่องจากการเรียกใช้ SHA256 CryptoService ใน .NET ครั้งแรกต้องใช้เวลาในการโหลด Library เข้าสู่หน่วยความจำก่อน แต่ช่วง 100k→1m ลดลงเหลือ 10.39 เท่า เพราะระบบโหลด Library เสร็จแล้ว การ Hash จึงทำงานได้สม่ำเสมอมากขึ้น ซึ่งสอดคล้องกับการทำงานของ Synchronous Script Component ที่เรียก ProcessInputRow() ทีละหนึ่งแถวอย่างต่อเนื่อง **[1]**

ขั้นตอนโหลดข้อมูล (Load) ช่วง 10k→100k เวลาเพิ่มขึ้น 5.92 เท่า แต่ช่วง 100k→1m เวลากลับลดลงเหลือเพียง 0.38 เท่า กล่าวคือที่ 1 ล้านแถวใช้เวลาน้อยกว่า 100,000 แถว จากการวิเคราะห์พบว่าอาจมีสาเหตุมาจากการตั้งค่า Batch Size = 5,000 แถวใน ODBC Destination ทำให้ส่งผลให้การโหลดข้อมูลขนาดใหญ่มีประสิทธิภาพสูงกว่าขนาดเล็ก อย่างไรก็ตามการระบุสาเหตุที่แน่ชัดต้องการการทดสอบและข้อมูลรองรับเพิ่มเติม

ภาพรวม (Total) เวลารวมเพิ่มขึ้น 16.54 เท่าในช่วงแรก และลดลงเหลือ 10.44 เท่าในช่วงหลัง ซึ่งสอดคล้องกับพฤติกรรมของ Transform เพราะ Transform เป็นขั้นตอนที่กำหนดเวลารวมทั้งหมด ดังที่เห็นในรูปที่ 3 ว่าเส้น Transform และเส้น Total แทบทับกันตลอด

References

[1]“Creating a Synchronous Transformation with the script component,” *Microsoft.com*. [Online]. Available: https://learn.microsoft.com/en-us/sql/integration-services/extending-packages-scripting-data-flow-script-component-types/creating-a-synchronous-transformation-with-the-script-component?view=sql-server-ver17

[2]rwestMSFT, “Memory management architecture guide - SQL Server,” *Microsoft.com*. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/memory-management-architecture-guide?view=sql-server-ver17